BLUEBIRD

A Hotel Management System

Our Hotel Management System solves the problem of manual hotel reservation and administration. It provides two main roles: users and admins. Users can register, log in, view hotel rooms, and submit reservations. Admins can manage bookings, confirm reservations, generate payment records, print invoices, manage rooms, manage staff, and view dashboard statistics. The project follows the SDLC phases from requirements analysis to design, implementation, testing, and maintenance.

Developed by:

	Name	*ID*
1	**Hossam Ahmed Adel**	**2305037**
2	**Antony Medhat Makram**	**2305065**
3	**Mohamed Ibrahim Hussein**	**2305026**
4	**Youssef Ahmed Wahid**	**2305105**
5	**Mayer Adel Kher**	**2305083**
6	**Rawan Sherif Ibrahim**	**2305144**

1. Introduction

Hotels need reliable systems to organize reservations, customer data, room records, payments, invoices, and staff data. Manual hotel administration can cause duplicate records, delayed confirmation, inaccurate payment calculation, and difficulty tracking room booking status. The Hotel Management System provides a centralized web application that reduces manual work and improves the speed and accuracy of hotel operations.

The system has two main user groups. Guests can access the public website, create accounts, log in, view hotel facilities, and submit booking requests. Admin or staff users can access the administrative panel to monitor the dashboard, confirm reservations, calculate payments, print invoices, manage available rooms, and manage staff records.

---

2. Project Description

The Hotel Management System is a PHP/MySQL web application. It uses a MySQL database named "bluebirdhotel" and contains modules for authentication, room booking, booking management, payment, invoice printing, room management, staff management, and dashboard statistics.

---

3. Project Objectives

- Allow customers to register, log in, and submit hotel room reservations.
- Allow admin/staff to manage reservations, rooms, staff, and payments.
- Automate payment calculation after booking confirmation.
- Generate printable invoices for confirmed bookings.
- Provide dashboard statistics that summarize reservations, rooms, staff, and profit.
- Document the complete development process using the phases and UML diagrams studied in the lectures.

---

4. Stakeholders and User Classes

Stakeholder/User Class	**Description**	**Main Interest**
Guest / Registered User	A hotel customer who uses the website to view facilities and reserve a room.	Easy booking, clear interface, and accurate reservation data.
Admin / Staff	Hotel employee who manages bookings, rooms, staff, payments, and invoices.	Efficient management and accurate operational records.
Hotel Management	Decision makers who monitor business status through reports and dashboard totals.	Reliable records, profit visibility, and operational control.
Development Team	Developers responsible for designing, implementing, testing, and documenting the system.	Clear requirements, maintainable code, correct diagrams, and successful project discussion.

5. Process Phases

Phase	Theoretical meaning	Project application
Requirements Analysis	Understand what the customer/user needs and capture data, functionality, and quality attributes.	Identified user/admin needs, system scope, data to store, functions, and quality requirements.
Specification	Describe exactly what the product will do, including inputs, outputs, constraints, and unambiguous requirements.	Prepared SRS-style details, use-case specifications, traceability, and sign-off placeholders.
Design	Describe how the system will be structured using high-level and detailed design plus UML models.	Prepared architecture, module design, database design, and required UML diagrams.
Implementation	Implement the detailed design in code and document the code and tested cases.	Used PHP, MySQL, HTML, CSS, Bootstrap, JavaScript, and XAMPP modules.
Integration	Integrate modules, test interfaces, and perform product/acceptance testing.	Integrated login, booking, confirmation, payment, room, staff, and invoice modules.
Maintenance	Apply changes after acceptance: adaptive, perfective, corrective, and preventive maintenance.	Outlined future fixes, security improvements, updates, backup, and regression testing.
Retirement	Replace or rewrite the system if it becomes unmaintainable or incompatible with a new environment.	Defined retirement triggers for the Hotel Management System.

---

6. Requirements Analysis Phase

6.1. Requirements Elicitation

Technique	How it was used in the project	Result
Prototyping	Screens and flows such as login, booking, admin dashboard, payment, and invoice pages were treated as a rapid prototype.	Improved understanding of what the system should provide before final documentation.
Observation	The common hotel workflow was observed conceptually: customer requests reservation, staff confirms, payment is calculated, and invoice is printed.	Produced the main workflow used in activity and sequence diagrams.
Interviews	Assumed questions were prepared for hotel staff/admin and customers, such as what data is needed for reservation and payment.	Identified fields such as name, email, phone, room type, bed, meal, check-in, check-out, and status.
Workshop / Team Discussion	Team members discussed scope and chose the manageable modules required for a working course project.	Defined included and excluded features.
Document Analysis	Existing PHP files and SQL database were reviewed to identify implemented functions and data tables.	Mapped implemented modules to requirements and test cases.

6.2. System Scope

Scope item	Description
In scope	User registration and login, admin/staff login, room browsing, reservation submission, booking list, booking confirmation, payment generation, invoice printing, room management, staff management, dashboard statistics, and export of booking data.
Out of scope	Online payment gateway, email/SMS notifications, multi-hotel branches, real-time payment integration, mobile application, and advanced role-based permissions.
System boundary	The application manages hotel booking data and administration records inside the bluebirdhotel database. It does not communicate with external banking or messaging services.

6.3. Data Requirements

Data entity	Stored data
User account	UserID, username, email, password.
Admin/staff login	Employee ID, employee email, employee password.
Room	Room ID, room type, bedding type.
Booking	Booking ID, customer name, email, country, phone, room type, bed, meal, number of rooms, check-in, check-out, number of days, status.
Payment	Payment ID, customer details, room type, bed, number of rooms, dates, number of days, room total, bed total, meal total, final total.
Staff	Staff ID, staff name, work type.

6.4. Functional Requirements

ID	Requirement	Actor	Priority
FR-01	The system shall allow a new user to sign up using username, email, password, and confirm password.	User	Must
FR-02	The system shall allow registered users to log in using email and password.	User	Must
FR-03	The system shall allow admin/staff to log in using employee email and password.	Admin/Staff	Must
FR-04	The system shall display hotel rooms and facilities to the user.	User	Must

FR-05	The system shall allow the user to submit room reservation details.	User	Must
FR-06	The system shall store each new booking with status NotConfirm until it is reviewed by admin.	System	Must
FR-07	The system shall allow admin to view all room bookings.	Admin/Staff	Must
FR-08	The system shall allow admin to confirm a booking.	Admin/Staff	Must
FR-09	The system shall calculate room, bed, meal, and final totals after booking confirmation.	System	Must
FR-10	The system shall store the payment record in the database after booking confirmation.	System	Must
FR-11	The system shall allow admin to print invoice details for a payment record.	Admin/Staff	Should
FR-12	The system shall allow admin to add and delete room records.	Admin/Staff	Should
FR-13	The system shall allow admin to add and delete staff records.	Admin/Staff	Should
FR-14	The system shall display dashboard statistics for bookings, rooms, staff, and profit.	Admin/Staff	Should
FR-15	The system shall allow admin to export booking data.	Admin/Staff	Could

6.5. Non-functional Requirements

ID / Quality Attribute	**Measurable requirement**	**Verification method**
NFR-01 Performance	Pages should load within 3 seconds on a standard local XAMPP environment for normal course-project data volume.	System testing with stopwatch or browser dev tools.
NFR-02 Security	Admin pages should be accessible only after successful admin/staff login session.	Try direct access to admin pages without session.
NFR-03 Usability	A new user should be able to complete a booking form without special training.	Usability test with at least one user following booking steps.
NFR-04 Reliability	A submitted valid booking should be stored correctly in roombook and should not lose required fields.	Compare form data with database record.
NFR-05 Availability	The system should be available while Apache and MySQL services are running.	Run XAMPP services and access the website.
NFR-06 Maintainability	Pages should be separated into user, admin, CSS, JavaScript, and database files to simplify maintenance.	Review project folder structure.
NFR-07 Scalability	The database should allow adding more rooms, bookings, staff, and payments without changing the main page structure.	Insert new records and verify they appear in admin pages.

6.6. SMART Requirements Review

SMART criterion	Application in this project
Specific	Requirements use clear verbs such as register, log in, submit booking, confirm booking, calculate payment, print invoice, add room, and delete staff.
Measurable	Non-functional requirements include measurable targets such as page load time and data storage verification.
Achievable	The scope is limited to a PHP/MySQL web application suitable for a 2-5 student course project.
Relevant	Each requirement supports a real hotel booking or administration activity.
Time-based	Requirements are documented and prioritized so the Must-Have features are completed before optional improvements.

6.7. MoSCoW Prioritization

Priority	Meaning	Hotel Management System requirements
Must-Have	Essential for a workable product.	User login, admin login, booking submission, booking storage, booking confirmation, payment calculation, database connection.
Should-Have	High value but not absolutely essential for first usable release.	Invoice printing, room management, staff management, dashboard statistics.
Could-Have	Useful if time is available.	Export booking data, improved search/filtering, richer reports.
Would-Have / Won't-Have now	Future wishes outside the current project release.	Online payment gateway, email/SMS confirmation, mobile app, multi-branch hotel support.

6.8. Ambiguity, Contradiction, and Incompleteness Review

Possible issue	Risk	Resolution in this report
"Manage booking" is ambiguous.	Could mean create, view, edit, confirm, delete, or search.	The report separates view bookings, confirm booking, delete booking, and edit booking into distinct functions.
"System should be fast" is vague.	Cannot be tested objectively.	NFR-01 states a measurable target of 3 seconds in local XAMPP for normal data volume.
Room availability can be incomplete.	Booking may be accepted even if room count is insufficient.	Availability checking is partially reflected in admin booking page; stronger validation is listed as future improvement.
Password security is incomplete.	Plain text passwords are a security weakness.	Future improvement: use password_hash() and password_verify().
Admin access control may be inconsistent.	Direct access may bypass intended navigation if session checks are missing.	Security testing and future role-based access control are included.

---

7. Specification Phase - Software Requirements Specification (SRS)

7.1. Product Perspective

The system is a standalone web-based hotel administration application running on a PHP/MySQL stack. Users access the website through a browser. Admin/staff access management functions through an admin dashboard. The MySQL database stores all persistent data.

7.2. Operating Environment

Component	Environment
Web server	Apache server through XAMPP.
Database server	MySQL/MariaDB, database name: bluebirdhotel.
Client	Modern browser such as Chrome, Edge, Firefox, or equivalent.
Programming language	PHP for backend logic; HTML, CSS, Bootstrap, and JavaScript for interface.

7.3. Operating Environment

Component	Environment
Web server	Apache server through XAMPP.
Database server	MySQL/MariaDB, database name: bluebirdhotel.
Client	Modern browser such as Chrome, Edge, Firefox, or equivalent.
Programming language	PHP for backend logic; HTML, CSS, Bootstrap, and JavaScript for interface.

7.4. Inputs, Processes, Outputs, and Constraints

Area	Specification
Inputs	Login credentials, signup details, customer contact data, room type, bed type, meal, number of rooms, check-in date, check-out date, room/staff entries, admin actions.
Processes	Credential validation, booking insertion, room count checking, status update, number-of-days calculation, payment total calculation, dashboard statistics, invoice display.
Outputs	Homepage, booking success/error messages, booking table, dashboard cards/charts, payment table, printable invoice, room list, staff list, exported booking data.
Constraints	Local PHP/MySQL deployment; no external payment gateway; admin actions depend on valid session and database connection; final report must include required UML diagrams.

7.5. Use Case Specifications

Use Case	Actor	Precondition	Main success scenario	Alternative/Exception
Register User	Guest	Guest is not logged in.	Guest enters username, email, password, and confirm password; system validates input and creates account.	If email exists or passwords do not match, system displays an error.
Login User	Registered User	User already has an account.	User enters email/password; system validates credentials; homepage opens.	If credentials are invalid, system displays login error.
Book Room	User	User is logged in.	User selects room type, bed, meal, number of rooms, dates, and submits; system stores booking as NotConfirm.	If required fields are missing, system asks for correction.
Confirm Booking	Admin/Staff	Admin is logged in and booking exists.	Admin clicks confirm; system updates status to Confirm and creates payment record.	If booking is already confirmed, system prevents duplicate confirmation.
Print Invoice	Admin/Staff	Payment record exists.	Admin opens invoice print page; system displays invoice and print option.	If payment record is missing, invoice cannot be generated.
Manage Rooms	Admin/Staff	Admin is logged in.	Admin adds or deletes room record.	Invalid room data should be rejected.

Manage Staff	Admin/Staff	Admin is logged in.	Admin adds or deletes staff record.	Invalid staff data should be rejected.

7.6. Main Success Scenario Example Using A/S Style

Step	Description
1	A: User enters email and password.
2	S: System validates user credentials against signup table.
3	S: System opens homepage for valid user.
4	A: User enters room booking details and submits.
5	S: System validates required booking fields.
6	S: System stores the booking in roombook with status NotConfirm.
7	A: Admin opens booking management page and clicks Confirm.
8	S: System updates booking status to Confirm.
9	S: System calculates room, bed, meal, and final totals.
10	S: System stores payment and allows invoice printing.

7.7. Requirements Traceability Matrix

Requirement ID	Design element / Module	Implementation file(s)	Test case
FR-01	Authentication module	index.php, signup table	TC-01
FR-02	Authentication module	index.php, signup table	TC-02
FR-03	Admin authentication module	index.php, emp_login table	TC-03
FR-05/FR-06	Booking module	home.php, roombook table	TC-04
FR-07/FR-08	Booking management module	admin/roombook.php, admin/roomconfirm.php	TC-05, TC-06
FR-09/FR-10	Payment module	admin/roomconfirm.php, admin/payment.php, payment table	TC-07
FR-11	Invoice module	admin/invoiceprint.php	TC-08
FR-12	Room management module	admin/room.php, room table	TC-09, TC-10
FR-13	Staff management module	admin/staff.php, staff table	TC-11, TC-12
FR-14	Dashboard module	admin/dashboard.php	TC-13

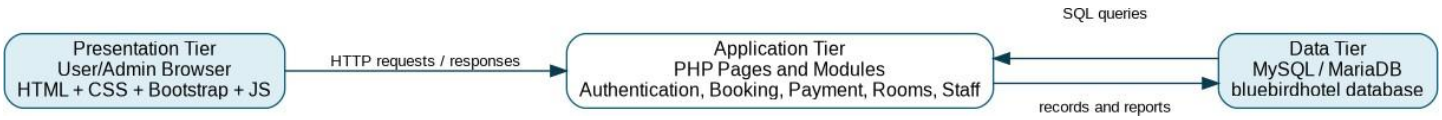
7.8. Requirements Sign-Off

Role	Name	Approval status	Signature / Date
Client / Instructor	NULL	Pending	NULL
Team Leader	Hossam Ahmed	Approved	Hossam Ahmed / 01-05-26
Developer 1	Mohamed Ibrahim	Approved	Mohamed Ibrahim / 28-4-26
Developer 2	Antony Medhat	Approved	Antony Medhat / 28-4-26

---

8. Design Phase

8.1. High-Level Architecture



8.2. Module Decomposition

Module	Responsibility	Main input	Main output
Authentication	User signup, user login, admin/staff login.	Email, password, username.	Authenticated session or error message.
Booking	Capture reservation details and store booking.	Customer data, room, bed, meal, dates.	Booking record with status NotConfirm.
Booking Management	Admin views, edits, confirms, or deletes bookings.	Booking ID and admin action.	Updated booking status or deleted record.
Payment	Calculate and display payment totals.	Confirmed booking data.	Payment record and final total.
Invoice	Display printable invoice.	Payment ID.	Invoice page.
Room Management	Add/delete room records.	Room type and bedding.	Updated room list.
Staff Management	Add/delete staff records.	Staff name and work.	Updated staff list.
Dashboard	Summarize business data.	Booking, payment, room, staff records.	Dashboard cards and charts.

8.3. Database Design

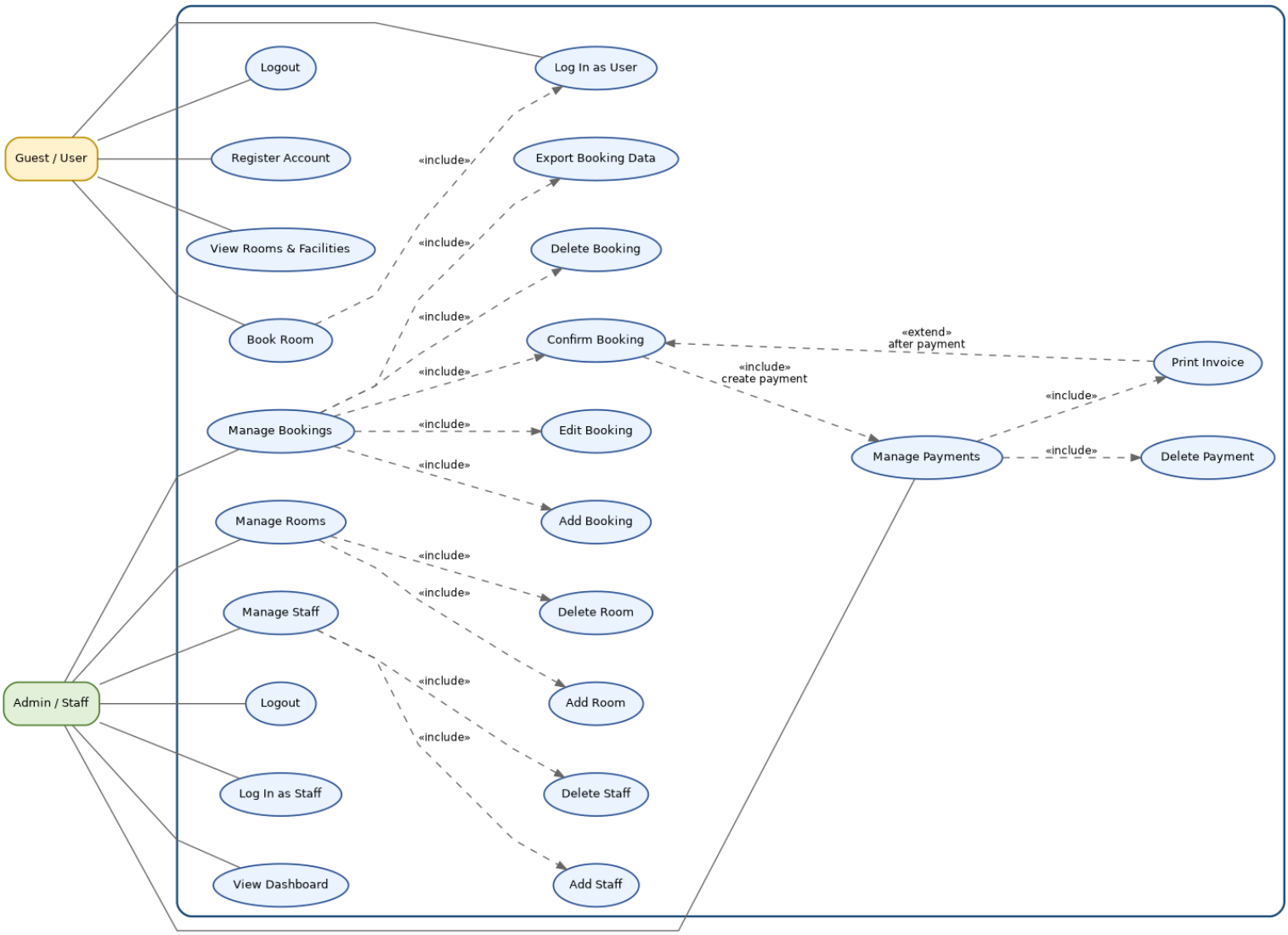
Table	Purpose	Main columns
signup	Stores user accounts.	UserID, Username, Email, Password
emp_login	Stores admin/staff login accounts.	empid, Emp_Email, Emp_Password
room	Stores available room type and bedding combinations.	id, type, bedding
roombook	Stores reservation details and booking status.	id, Name, Email, Country, Phone, RoomType, Bed, Meal, NoofRoom, cin, cout, nodays, stat
payment	Stores calculated payment and invoice-related totals.	id, Name, Email, RoomType, Bed, NoofRoom, cin, cout, noofdays, roomtotal, bedtotal, meal, mealtotal, finaltotal
staff	Stores staff records.	id, name, work

8.4. Design Decisions

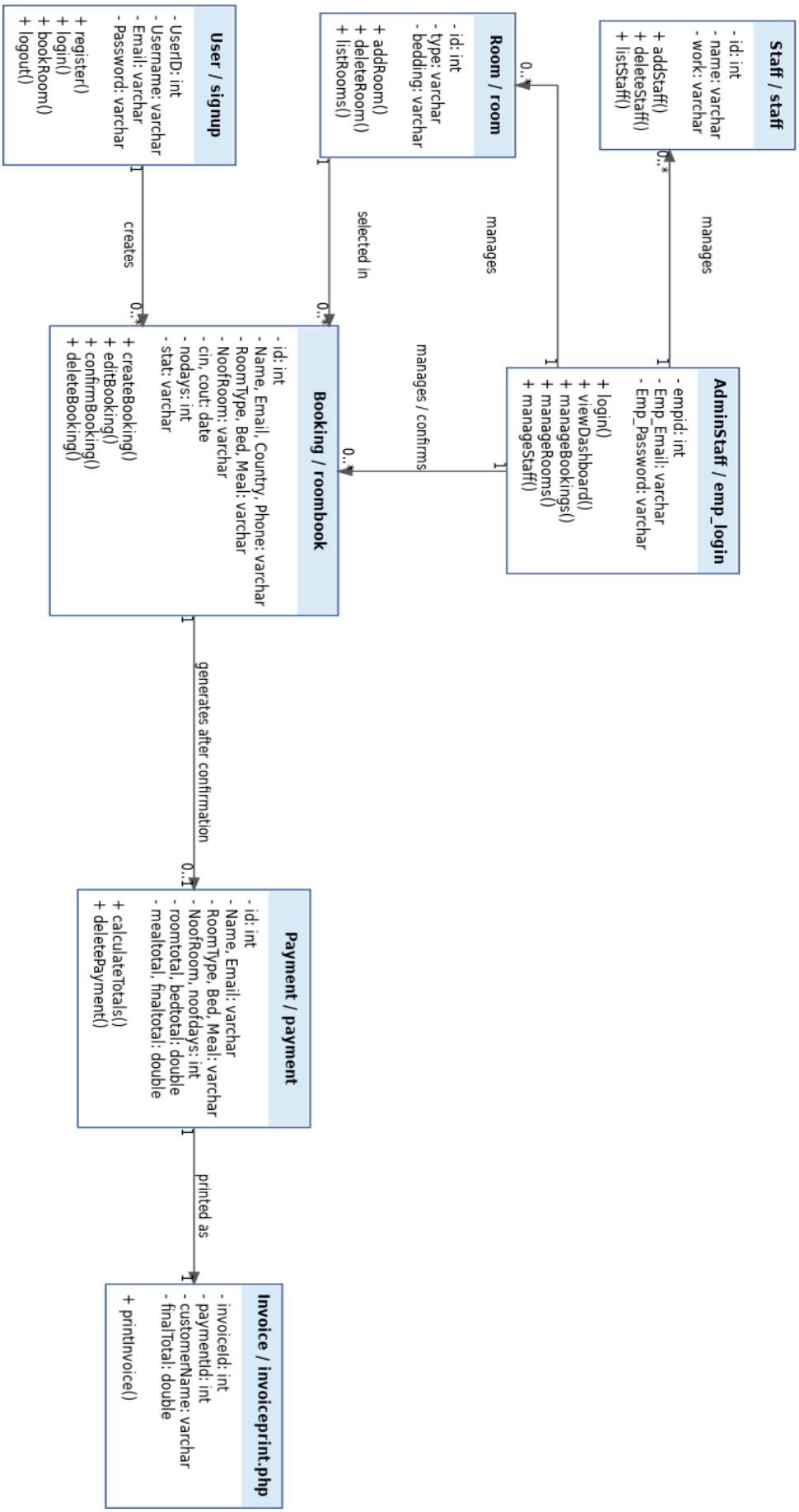
- A PHP/MySQL architecture was selected because it is simple to deploy using XAMPP and suitable for the course project scope.
- Separate admin pages were used for dashboard, booking, payment, room, and staff modules to improve maintainability.
- Bookings start with status NotConfirm and move to Confirm only after admin review. This supports the state machine model.
- Payment is generated after booking confirmation because it depends on booking details such as room type, bed, meal, number of rooms, and number of days.
- The database uses separate tables for users, staff/admin login, rooms, bookings, payments, and staff to keep data organized.

8.5. Diagrams

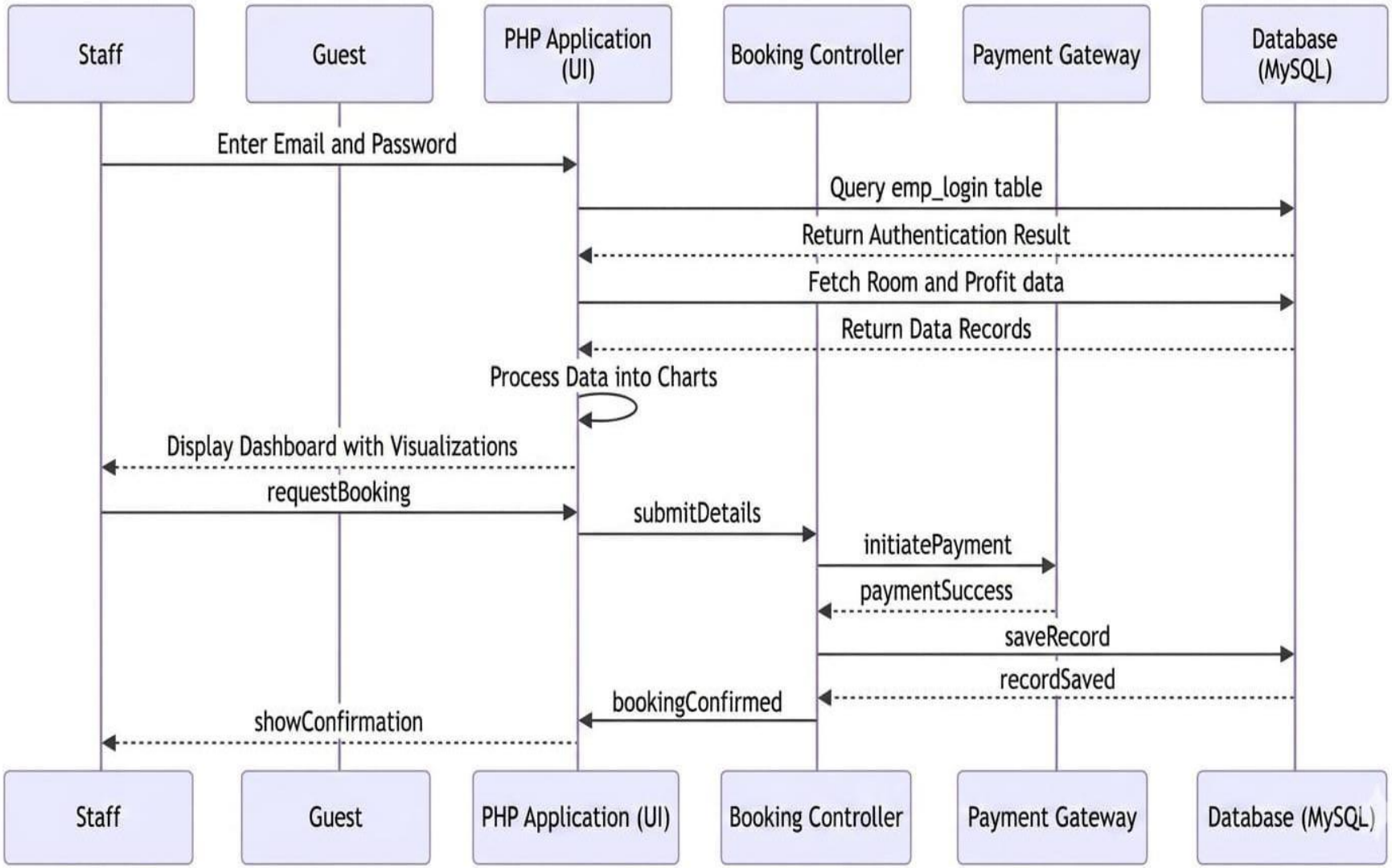
Use Case Diagram



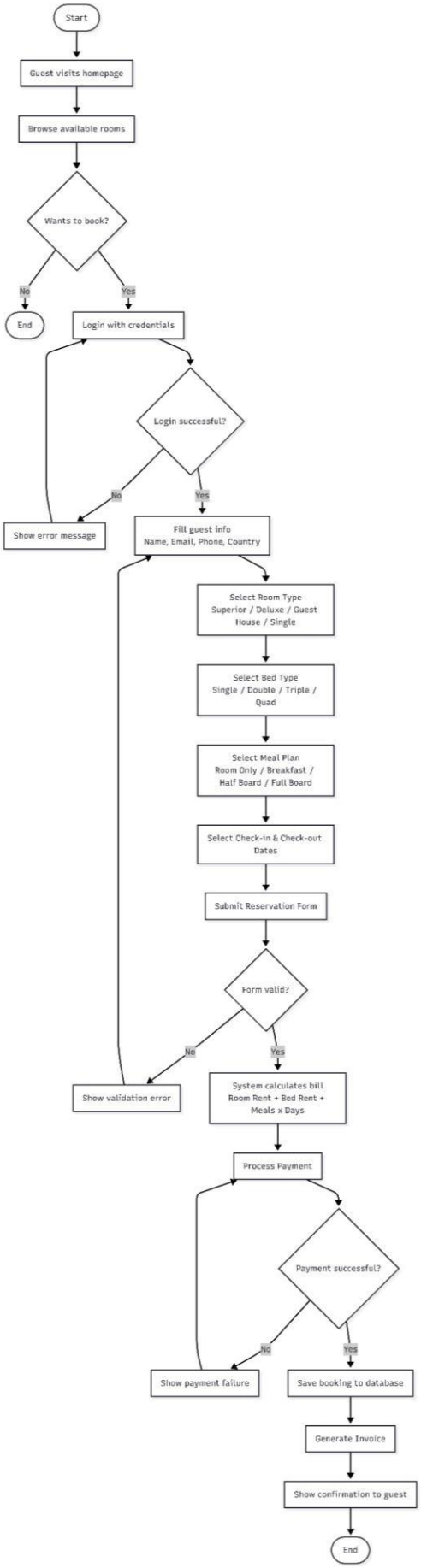
Class Diagram



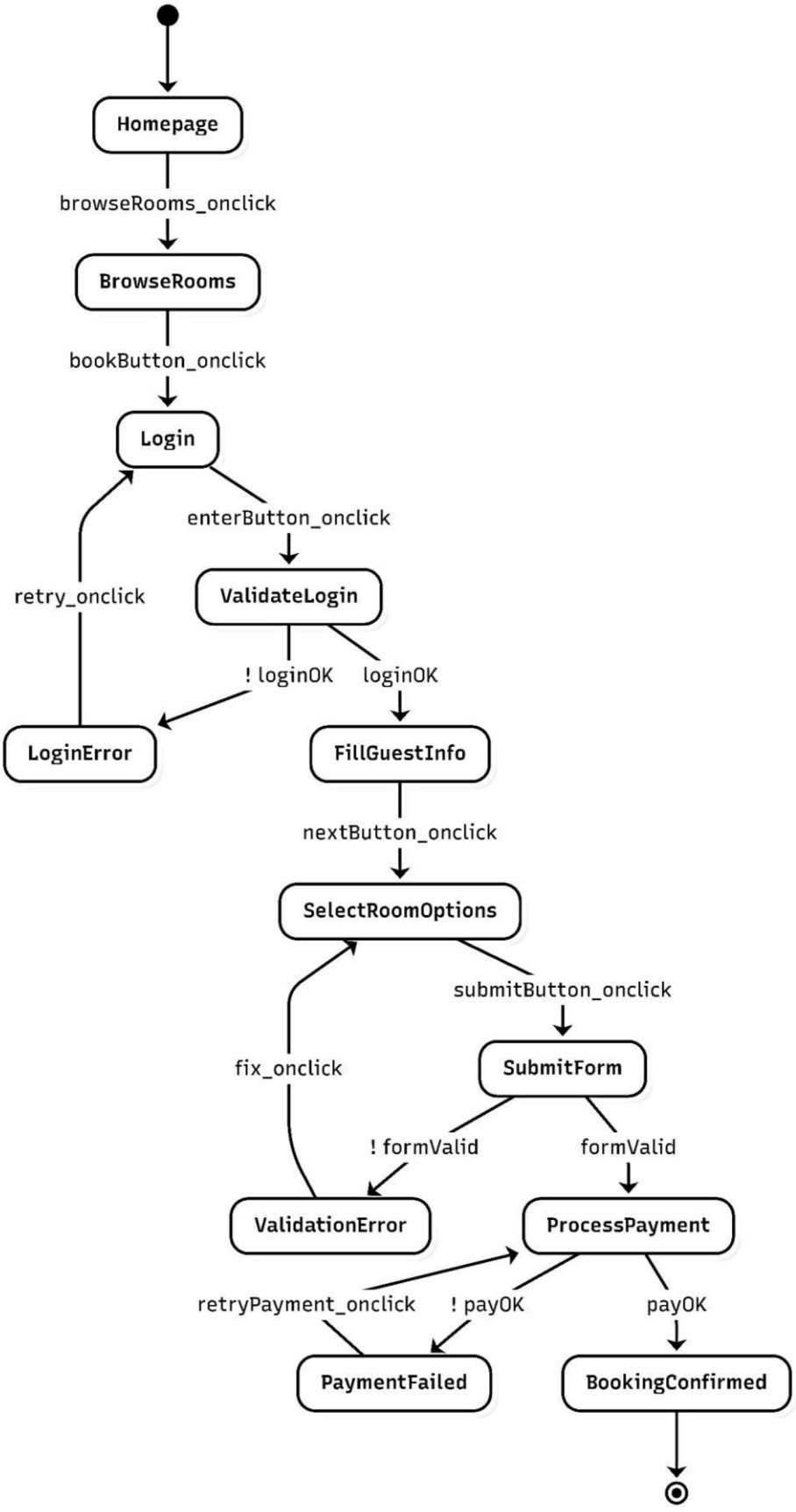
Sequence Diagram



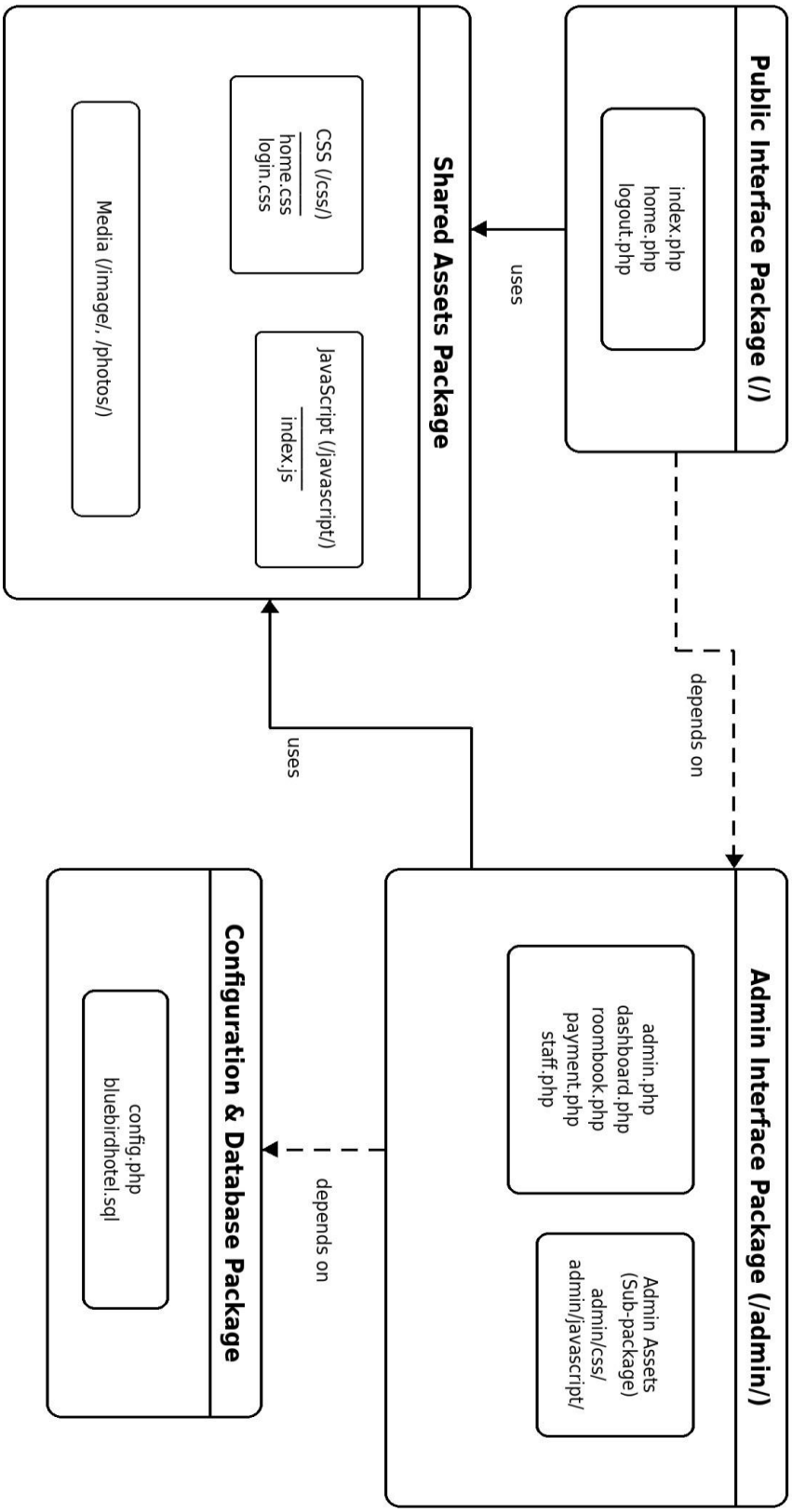
Activity Diagram



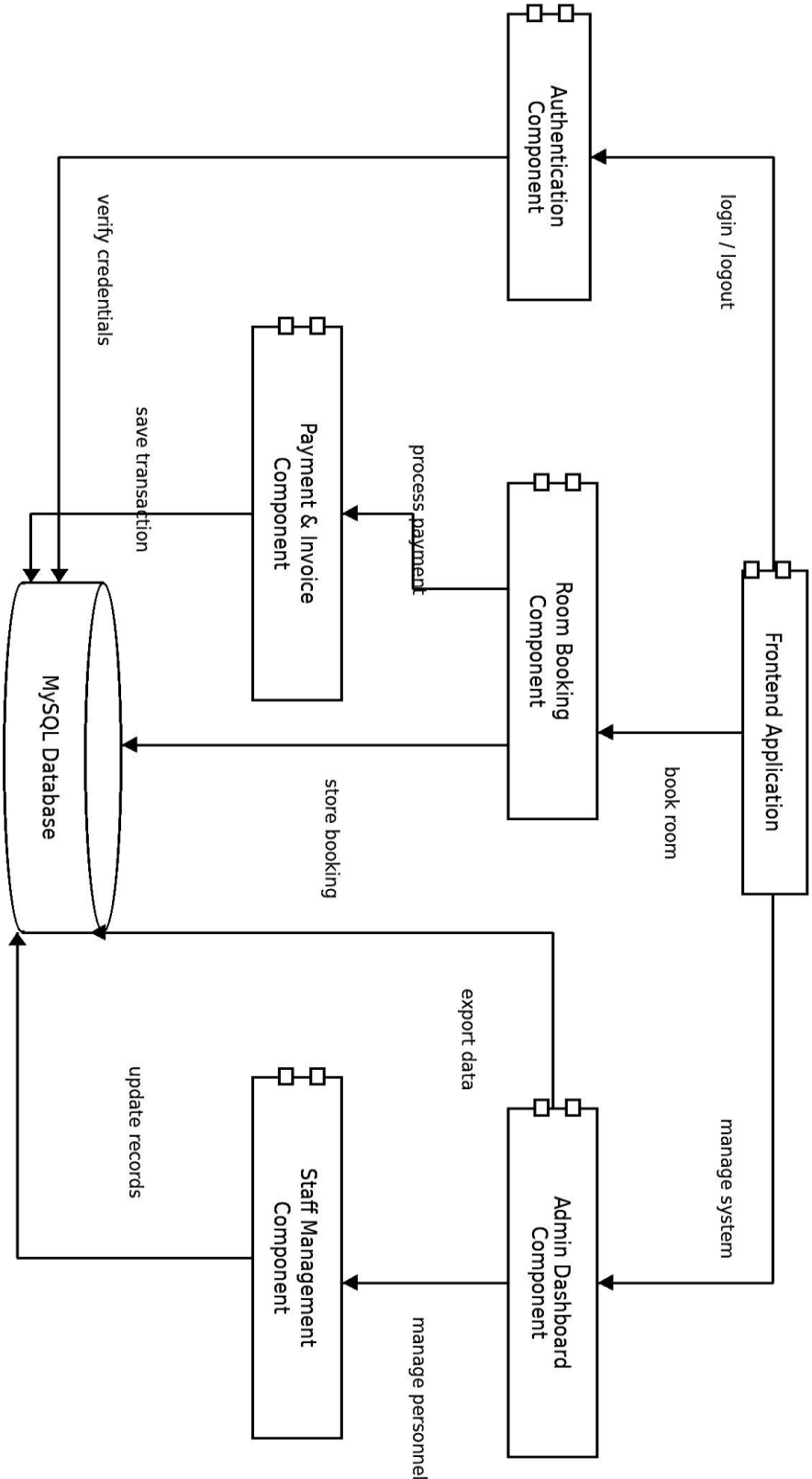
State Diagram



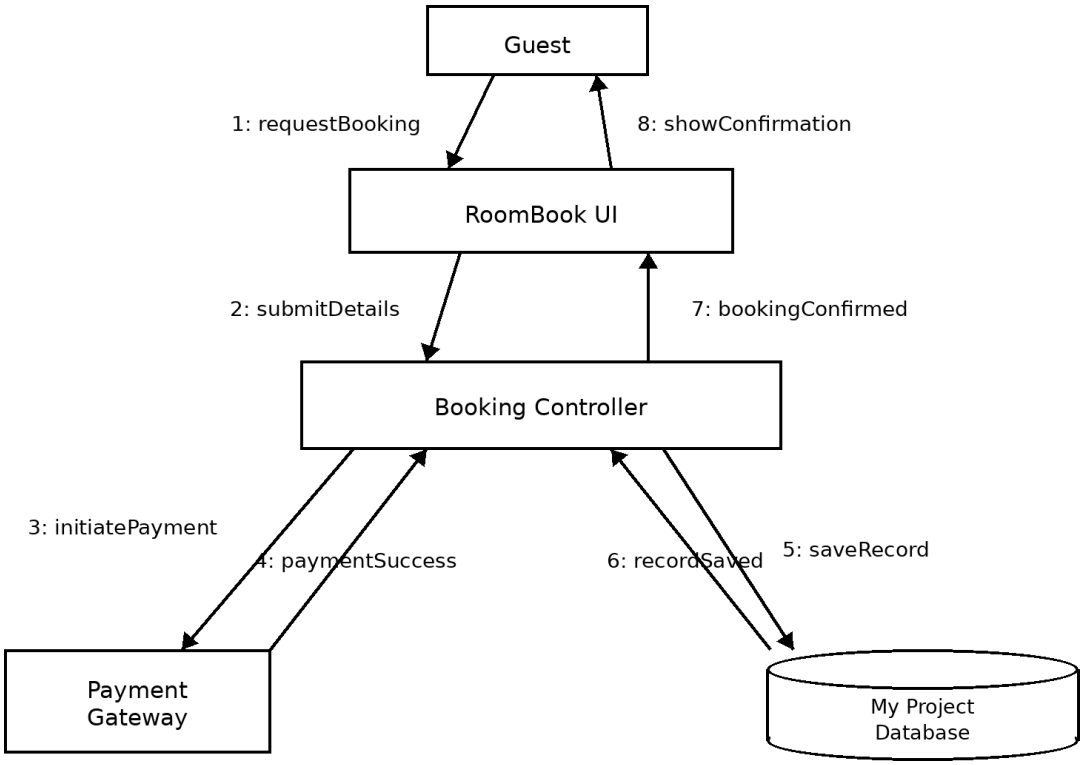
Package Diagram



Component Diagram



Collaboration Diagram



9. Implementation Phase

9.1. Technologies and Tools Used

Technology / Tool	Usage in the project
PHP	Backend logic, session handling, database queries, form processing.
MySQL / MariaDB	Persistent storage through the bluebirdhotel database.
HTML	Structure of user and admin pages.
CSS	Custom styling for login and home pages.
Bootstrap	Responsive layout and interface components.
JavaScript	Client-side page interaction and table search.
XAMPP	Local Apache and MySQL development environment.
phpMyAdmin	Database creation, import, and inspection.
Draw.io / StarUML / Graphviz equivalent	Computerized UML diagrams.

9.2. Source Code Organization

Path/File	Purpose
index.php	User login, user signup, and admin/staff login.
home.php	User homepage and booking form.
config.php	Database connection settings.
admin/admin.php	Admin layout with dashboard frame links.
admin/dashboard.php	Dashboard statistics and charts.
admin/roombook.php	Booking management page.
admin/roomconfirm.php	Booking confirmation and payment insertion logic.
admin/payment.php	Payment table and search.
admin/invoiceprint.php	Invoice printing page.
admin/room.php	Room add/delete management.
admin/staff.php	Staff add/delete management.
bluebirdhotel.sql	Database schema and sample data.

9.3. Implementation Development Steps

- Set up XAMPP and create the bluebirdhotel database.
- Import bluebirdhotel.sql using phpMyAdmin.
- Configure database connection in config.php.
- Implement and test user signup and login.
- Implement user homepage and booking submission to roombook.
- Implement admin dashboard and booking management pages.
- Implement booking confirmation, payment calculation, and payment insertion.
- Implement invoice printing, room management, staff management, and export functions.
- Perform syntax checking, manual test cases, integration testing, and documentation.

9.4. Coding Practice Notes

Practice	**Current status / recommendation**
Separation of concerns	The project separates user pages, admin pages, stylesheets, JavaScript, images, and database script.
Database connection reuse	config.php centralizes the MySQL connection.
Session handling	Sessions are used to distinguish logged-in users/admins.
Input validation	Basic required-field checks exist; deeper validation should be added for dates, phone, and room availability.
Security	Some login queries use prepared statements; all remaining direct SQL queries should be converted to prepared statements.
Password storage	Passwords should be hashed in future using password_hash() and verified with password_verify().

---

10. Testing

10.1. Testing Strategy

Testing type	**Purpose**	**Example in this project**
Desk checking / programmer testing	Informal checking while writing code.	Review PHP conditions, SQL queries, and form field names.
Unit testing	Check individual pages/modules.	Test user login, signup, add room, add staff separately.
Integration testing	Check module interfaces.	Submit booking then verify it appears in admin booking table.
System testing	Check the complete system behavior.	Run full flow from signup to invoice printing.
Acceptance testing	Decide whether the delivered system satisfies the project objective.	Final team/instructor discussion and demonstration.
Regression testing	Retest old functions after changes.	After changing payment logic, retest login, booking, confirmation, and invoice.

10.2. Test Cases

Test ID	Scenario	Steps	Expected result	Status
TC-01	User signup	Enter username, email, password, confirm password.	Account is created and user can log in.	Verified
TC-02	User login	Enter valid user email and password.	User homepage opens.	Verified
TC-03	Admin login	Enter valid admin email and password.	Admin panel opens.	Verified
TC-04	Submit booking	User fills booking form and submits.	Booking is saved in roombook with NotConfirm status.	Verified
TC-05	View bookings	Admin opens booking management page.	All bookings appear in table.	Verified
TC-06	Confirm booking	Admin clicks Confirm for a NotConfirm booking.	Booking status becomes Confirm.	Verified
TC-07	Generate payment	Confirm a booking.	Payment record is created with final total.	Verified
TC-08	Print invoice	Open invoice print page for a payment record.	Invoice displays customer and total payment details.	Verified
TC-09	Add room	Admin chooses room type and bed then clicks Add Room.	Room record appears in room list.	Verified
TC-10	Delete room	Admin clicks Delete on a room record.	Room record is removed.	Verified
TC-11	Add staff	Admin enters staff name and work then clicks Add Staff.	Staff record appears in staff list.	Verified
TC-12	Delete staff	Admin clicks Delete on a staff record.	Staff record is removed.	Verified
TC-13	Dashboard statistics	Admin opens dashboard.	Counts and totals are displayed.	Verified
TC-14	Invalid login	Enter wrong password.	System shows invalid credentials / access is not granted.	Verified

---

11. Maintenance Phase

Maintenance type	Meaning	Project application
Corrective	Diagnose and fix discovered errors.	Fix wrong table heading, failed redirect, duplicate payment generation, or incorrect total calculation.
Adaptive	Modify the system for environment changes.	Update code for a newer PHP/MySQL version or different hosting server.
Perfective	Add or improve functionality based on user needs.	Add online payment, email confirmation, room availability calendar, or monthly reports.
Preventive	Improve maintainability/reliability before failure occurs.	Use prepared statements everywhere, hash passwords, refactor repeated code, and add backups.

11.1. Future improvements

- Use password hashing for all user and admin passwords.
- Convert all direct SQL queries to prepared statements to reduce SQL injection risk.
- Add stricter validation for dates, phone numbers, number of rooms, and room availability.
- Add customer booking history and booking cancellation flow.
- Add email or SMS confirmation after reservation and payment.
- Add online payment gateway integration.
- Add role-based access control for different staff responsibilities.
- Add monthly/yearly financial reports and downloadable invoices.
- Improve mobile responsiveness of admin pages.
- Deploy the system on a live hosting environment with HTTPS.

11.2. Regression Testing Plan

After each maintenance change, the team should rerun the main regression suite: user login, booking submission, admin login, booking confirmation, payment generation, invoice printing, add/delete room, add/delete staff, and dashboard statistics.

---

12. Retirement

when the software may become unsuitable for further maintenance. For this project, retirement does not mean immediate deletion. It means replacing, rewriting, or migrating the system when maintenance is no longer practical.

Retirement trigger	Project Example	Recommended action
Drastic design change	The hotel needs a multi-branch cloud system with advanced roles and payment integration.	Rewrite using a more scalable architecture or framework.
New environment	The system must run on a different platform or modern hosting stack not compatible with the current code.	Migrate and refactor code.
Missing/inaccurate documentation	Future developers cannot understand the current system behavior.	Update documentation or rewrite core modules.
Cheaper to rewrite than modify	Security and scalability changes become too large for the current procedural PHP structure.	Start a new version using a modern framework and preserve database migration plan.

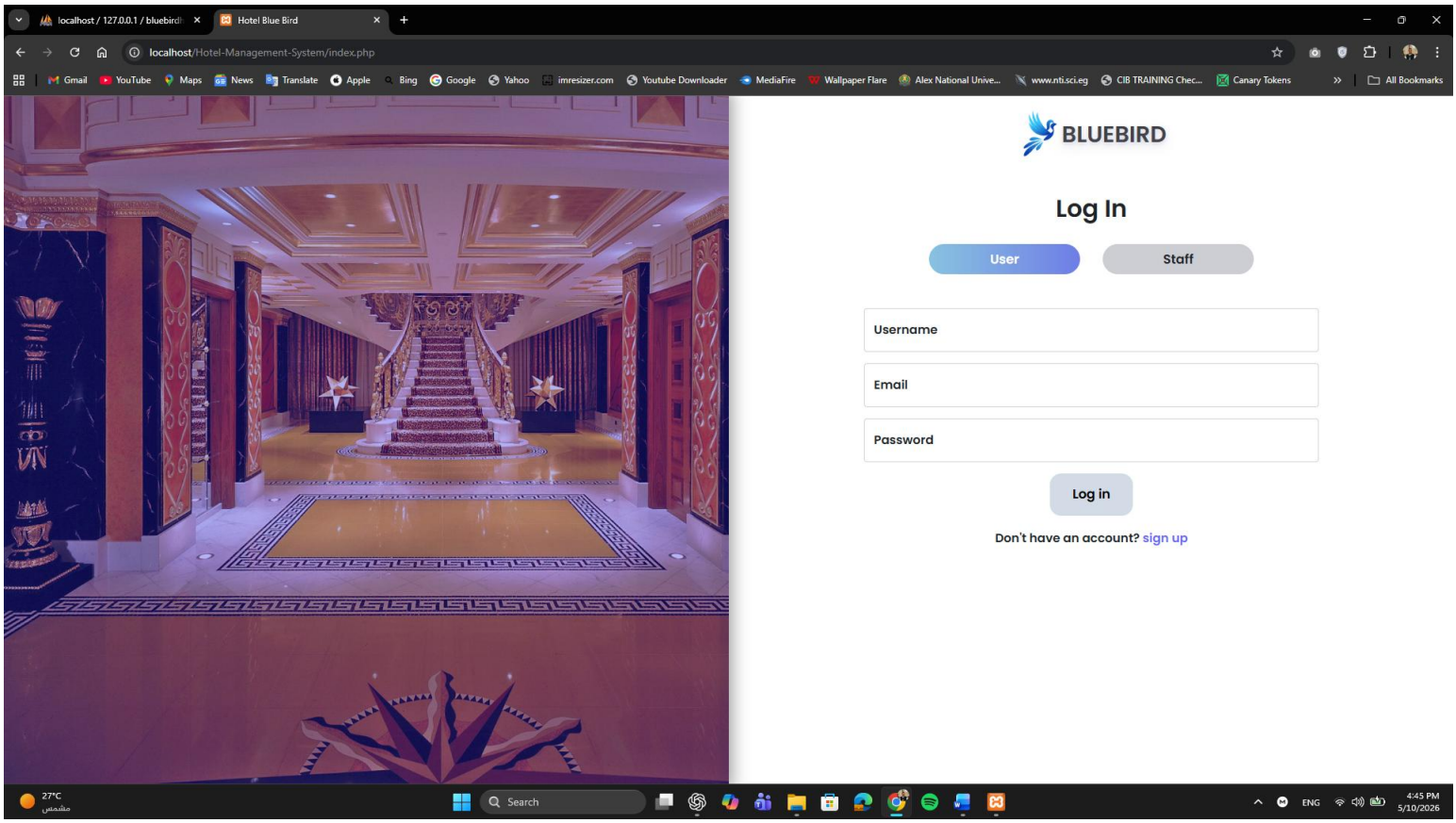
Appendix A. Installation Guide

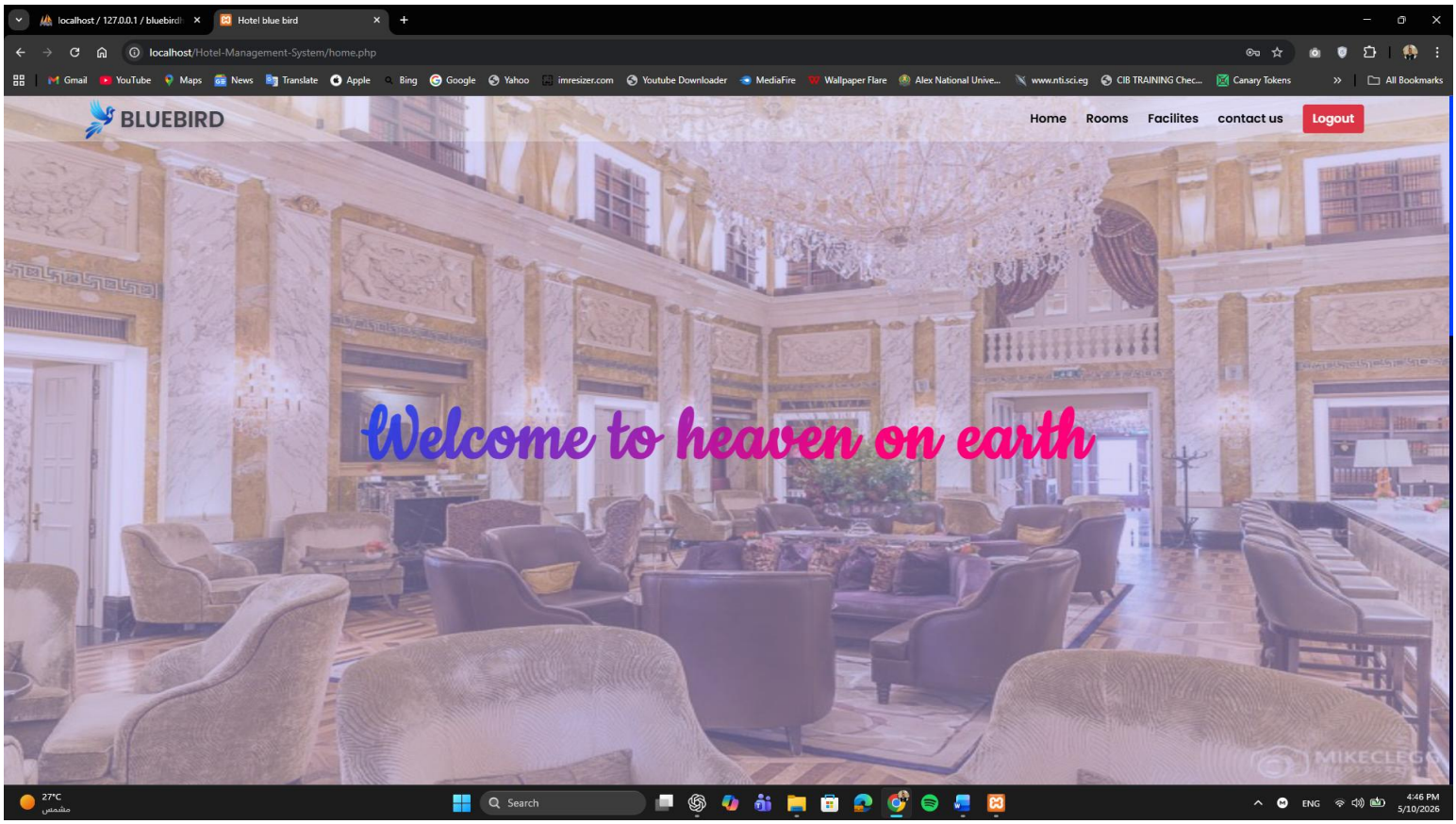
Step	Action
1	Install **XAMPP** and start Apache and **MySQL**.
2	Copy the project folder to “C:\xampp\htdocs\Hotel-Management-System”
3	Open http://localhost/phpmyadmin.
4	Create a database named **bluebirdhotel**.
5	Import **bluebirdhotel.sql** into the **bluebirdhotel** database.
6	Verify **config.php** uses server localhost, username root, empty password, and database bluebirdhotel.
7	Open http://localhost/Hotel-Management-System/index.php.
8	Log in using user/admin test accounts and execute the test cases.

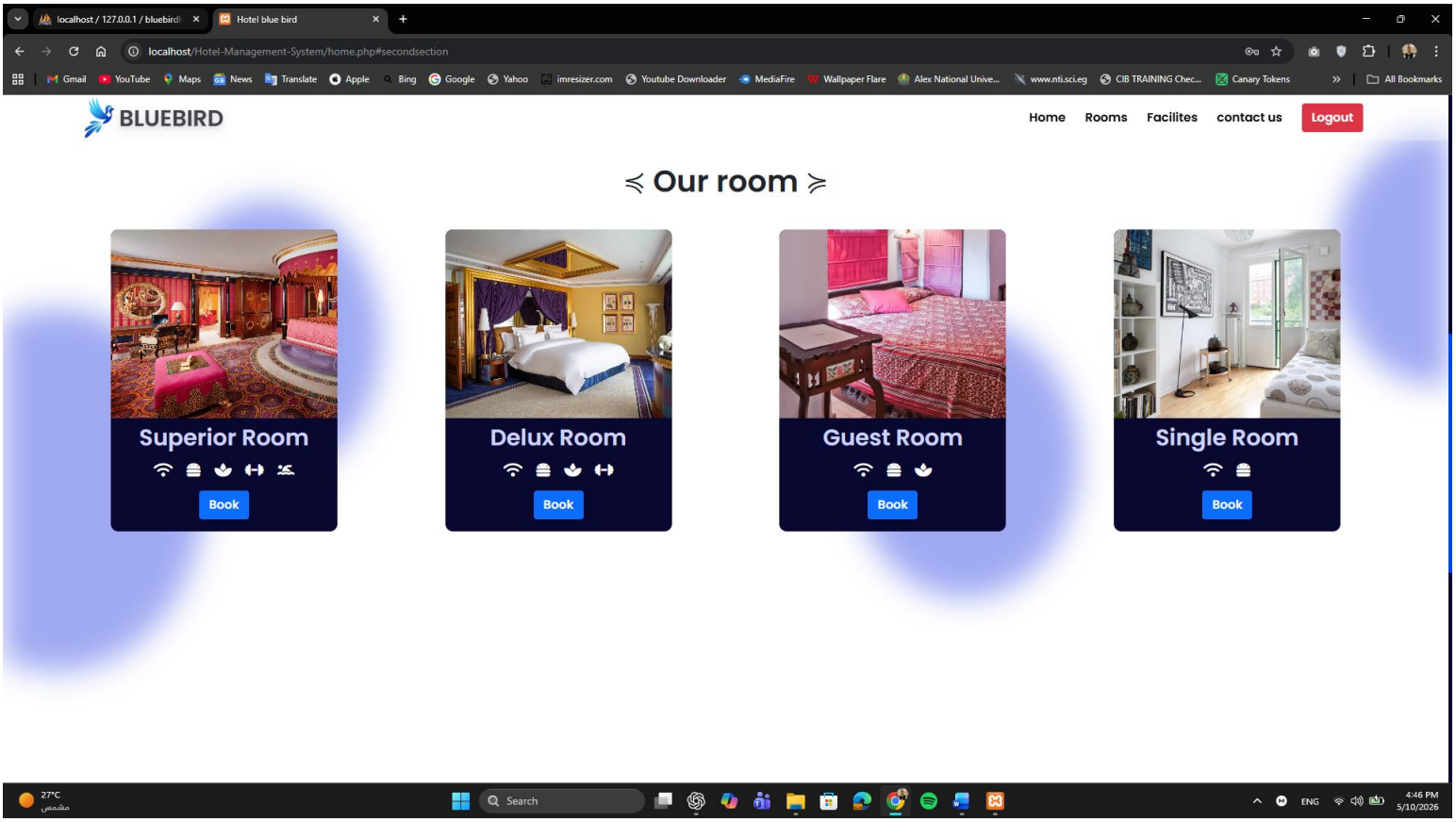
Appendix B. Test Accounts

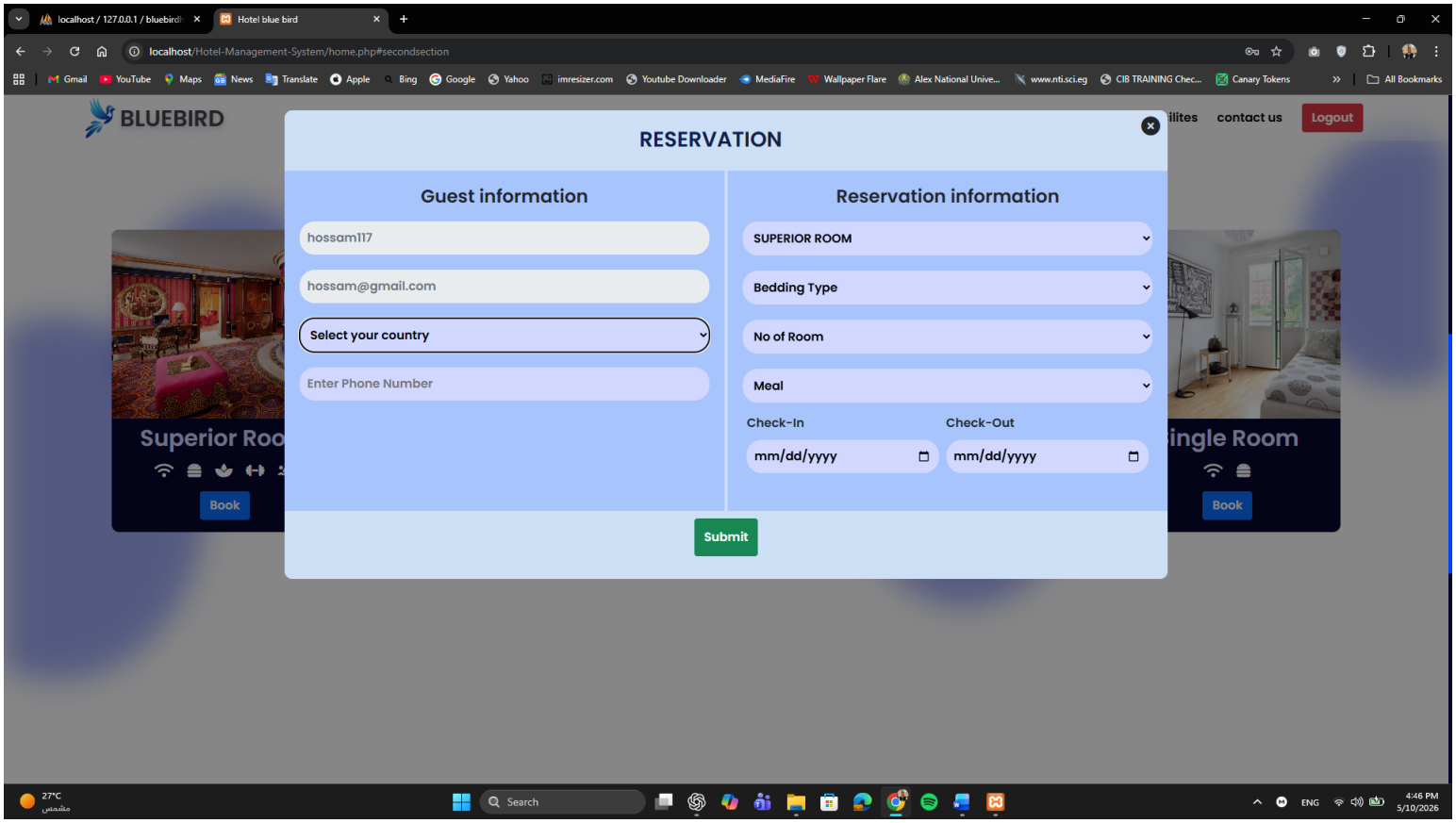
Role	Email	Password
Admin/Staff	admin@gmail.com	1234
User	test@gmail.com	123
User	mohamed@gmail.com	mohamed@gmail.com

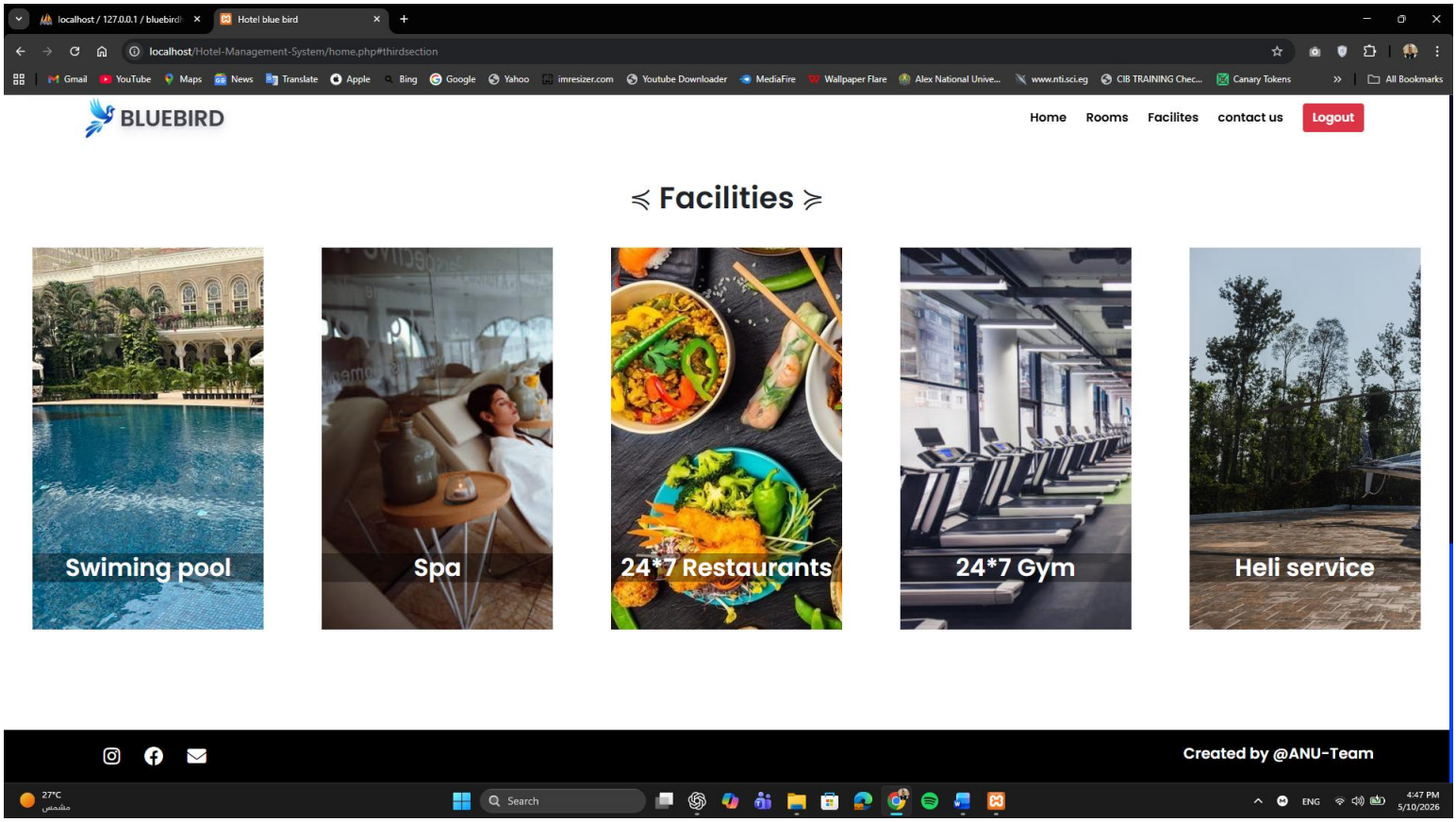
Screenshots:

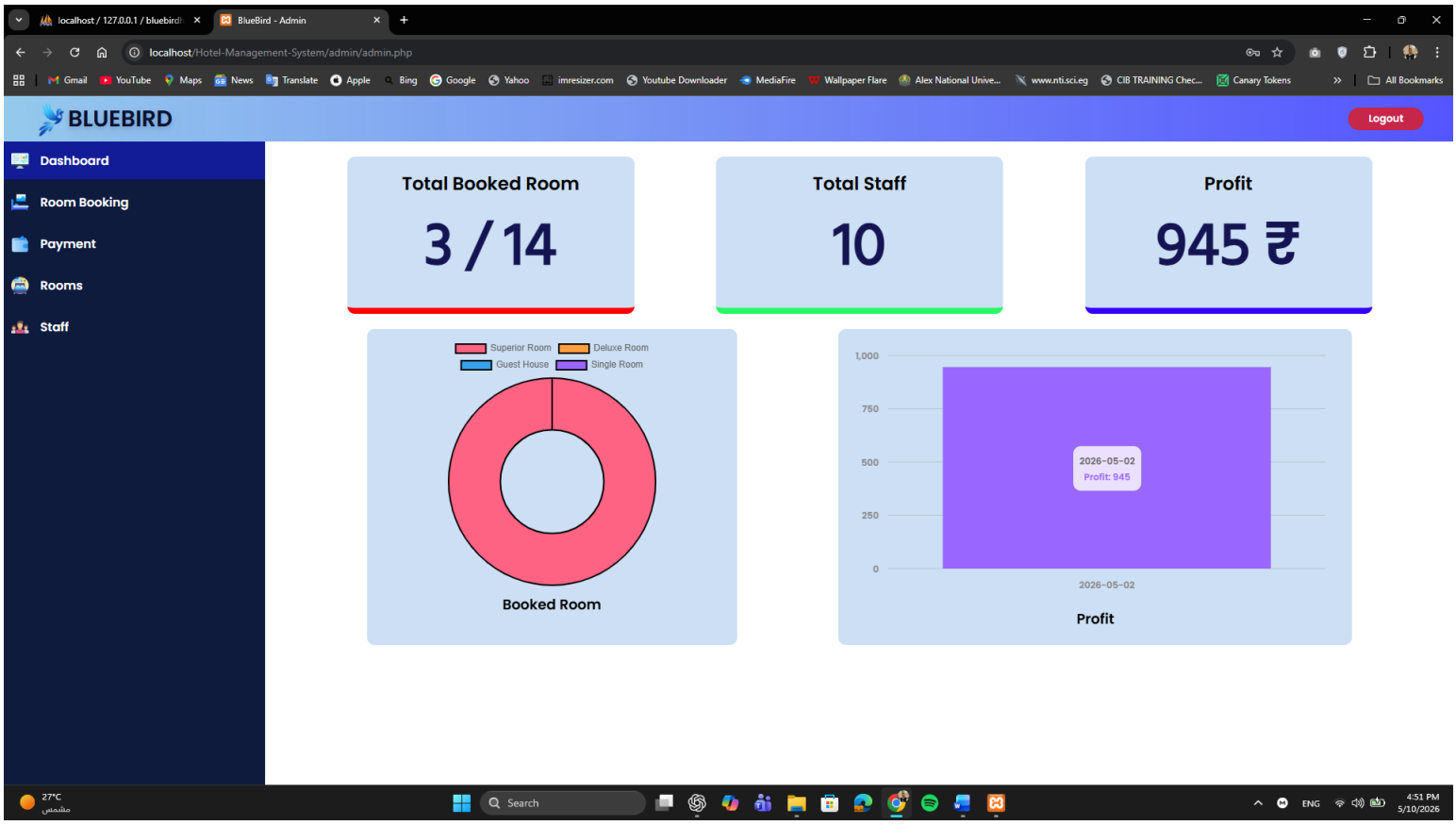


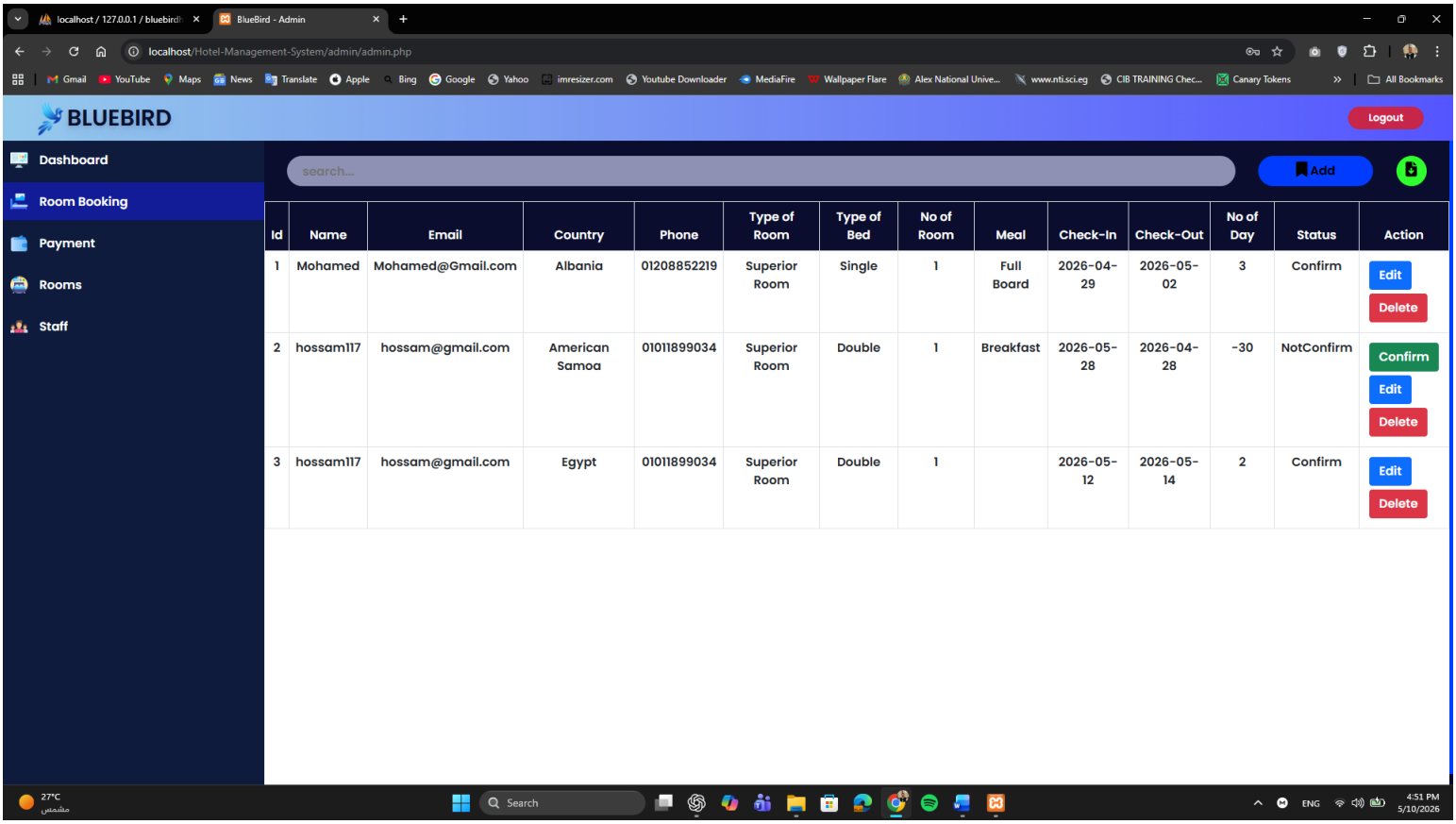



BLUEBIRD
Home
Rooms
Facilites
contact us
Logout
Our room
Superior Room
Book
Delux Room
Book
Guest Room
Book
Single Room
Book

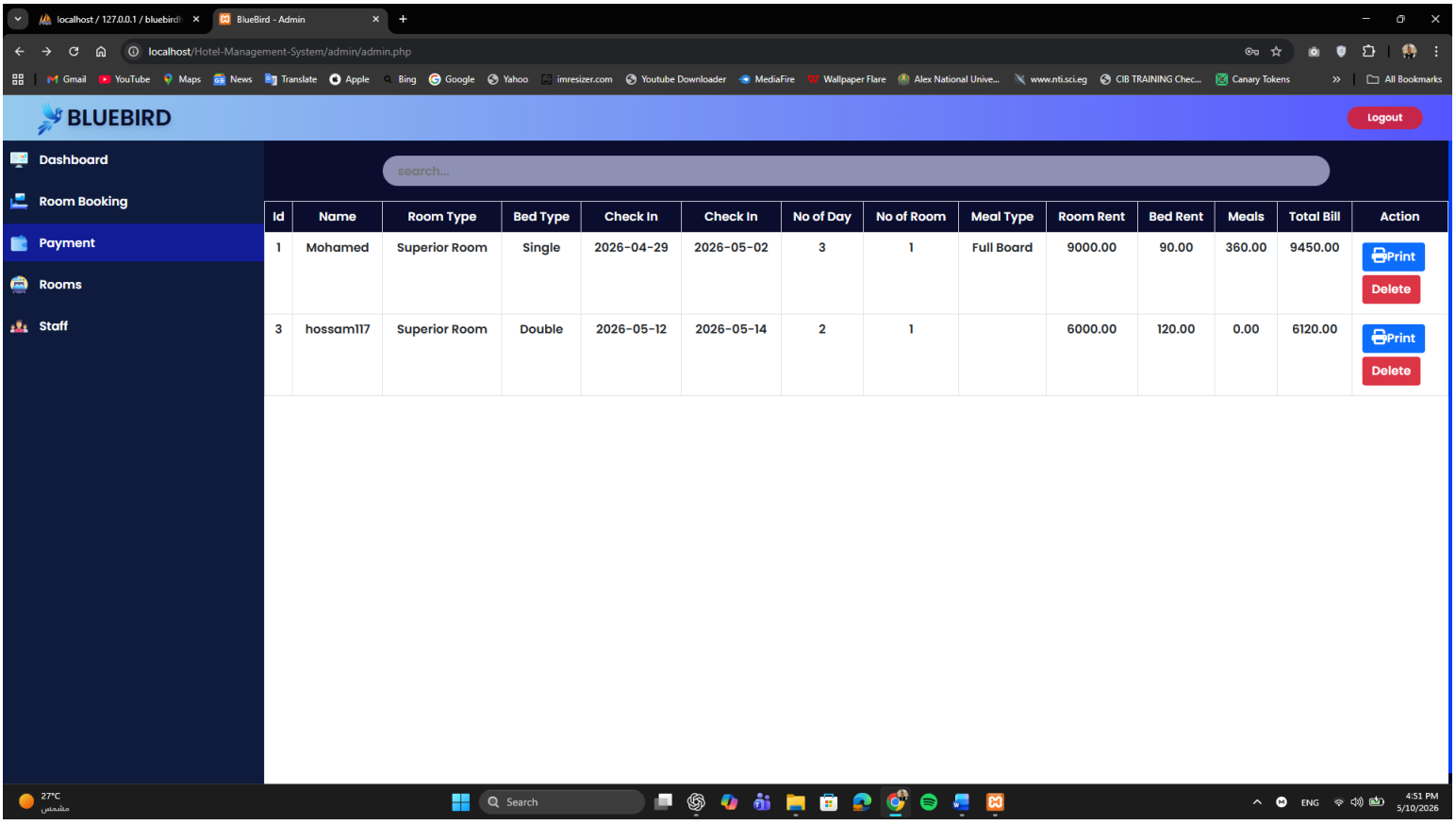

BLUEBIRD
contact us
Logout
RESERVATION
Guest information
hossam117
hossam@gmail.com
Select your country
Enter Phone Number
Reservation information
SUPERIOR ROOM
Bedding Type
No of Room
Meal
Check-In
mm/dd/yyyy
Check-Out
mm/dd/yyyy
Submit
Book
Book


localhost / 127.0.0.1 / bluebird
Hotel blue bird
localhost/Hotel-Management-System/home.php#thirdsection
BLUEBIRD
Home
Rooms
Facilites
contact us
Logout
Facilities
Swiming pool
Spa
24*7 Restaurants
24*7 Gym
Heli service
Created by @ANU-Team

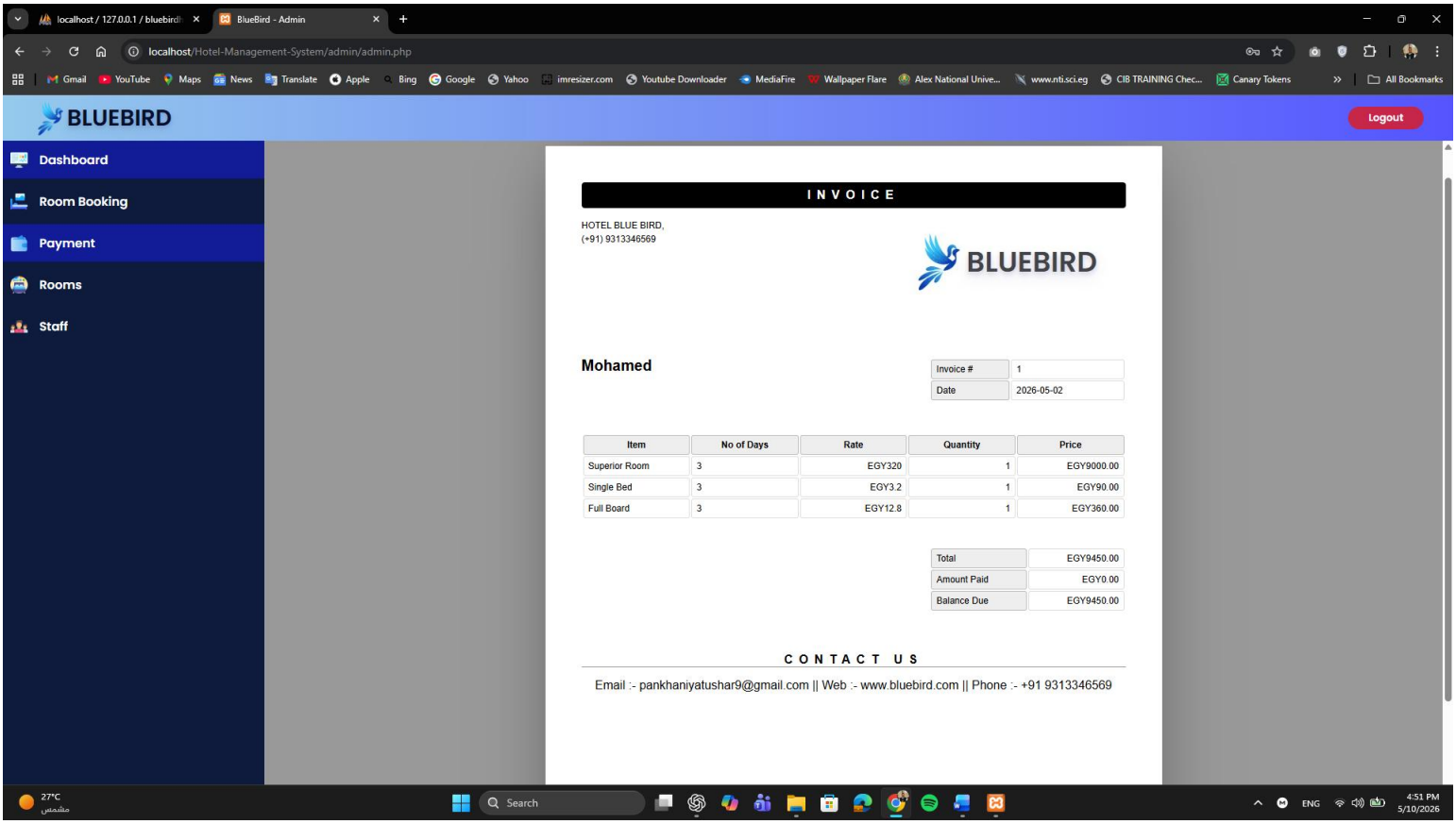

localhost / 127.0.0.1 / bluebird
BlueBird - Admin
localhost/Hotel-Management-System/admin/admin.php
BLUEBIRD
Logout
Dashboard
Room Booking
Payment
Rooms
Staff
Total Booked Room
3 / 14
Total Staff
10
Profit
945 ₹
Superior Room
Deluxe Room
Guest House
Single Room
Booked Room
1,000
750
500
250
0
2026-05-02
Profit: 945
2026-05-02
Profit

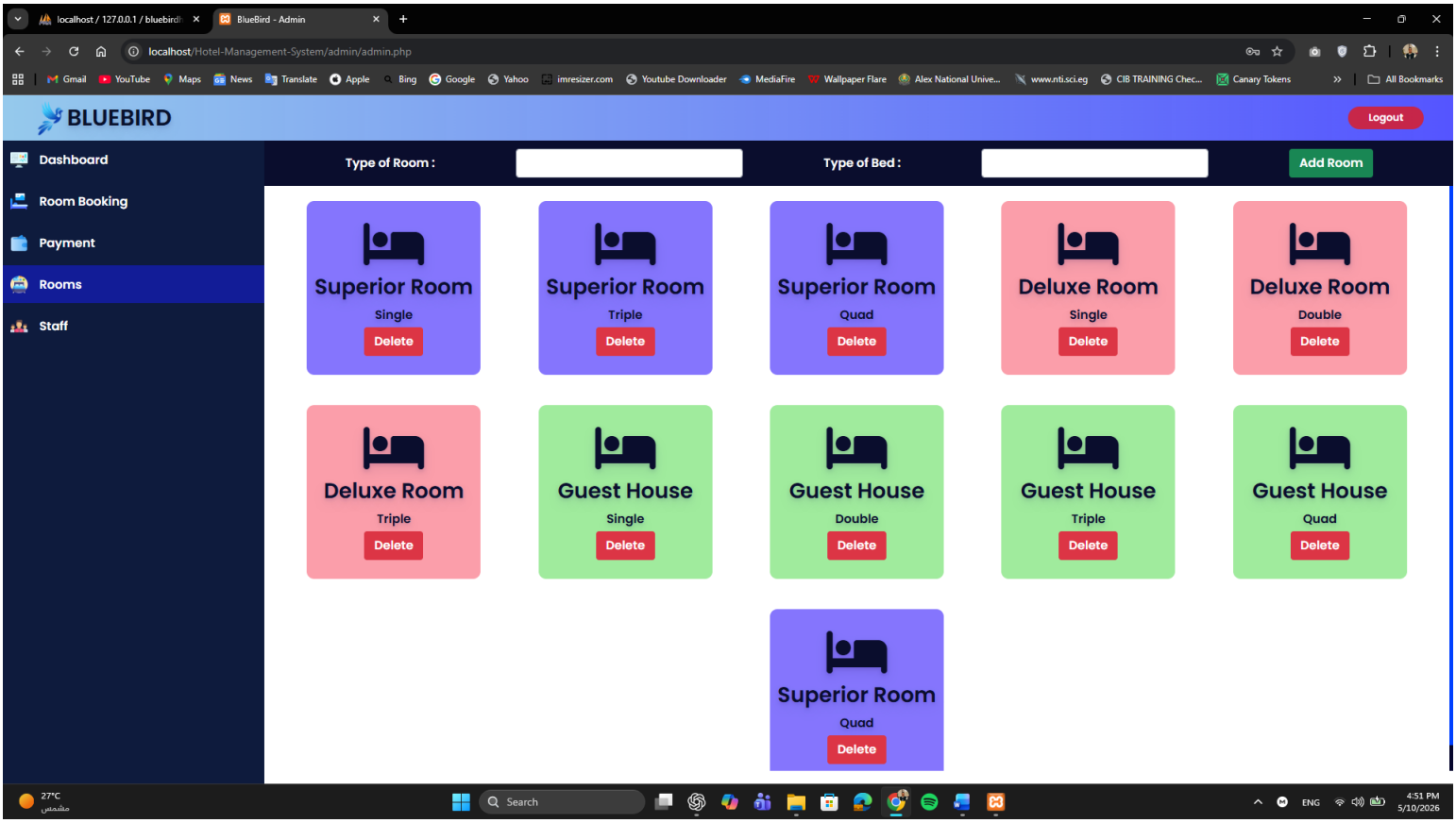


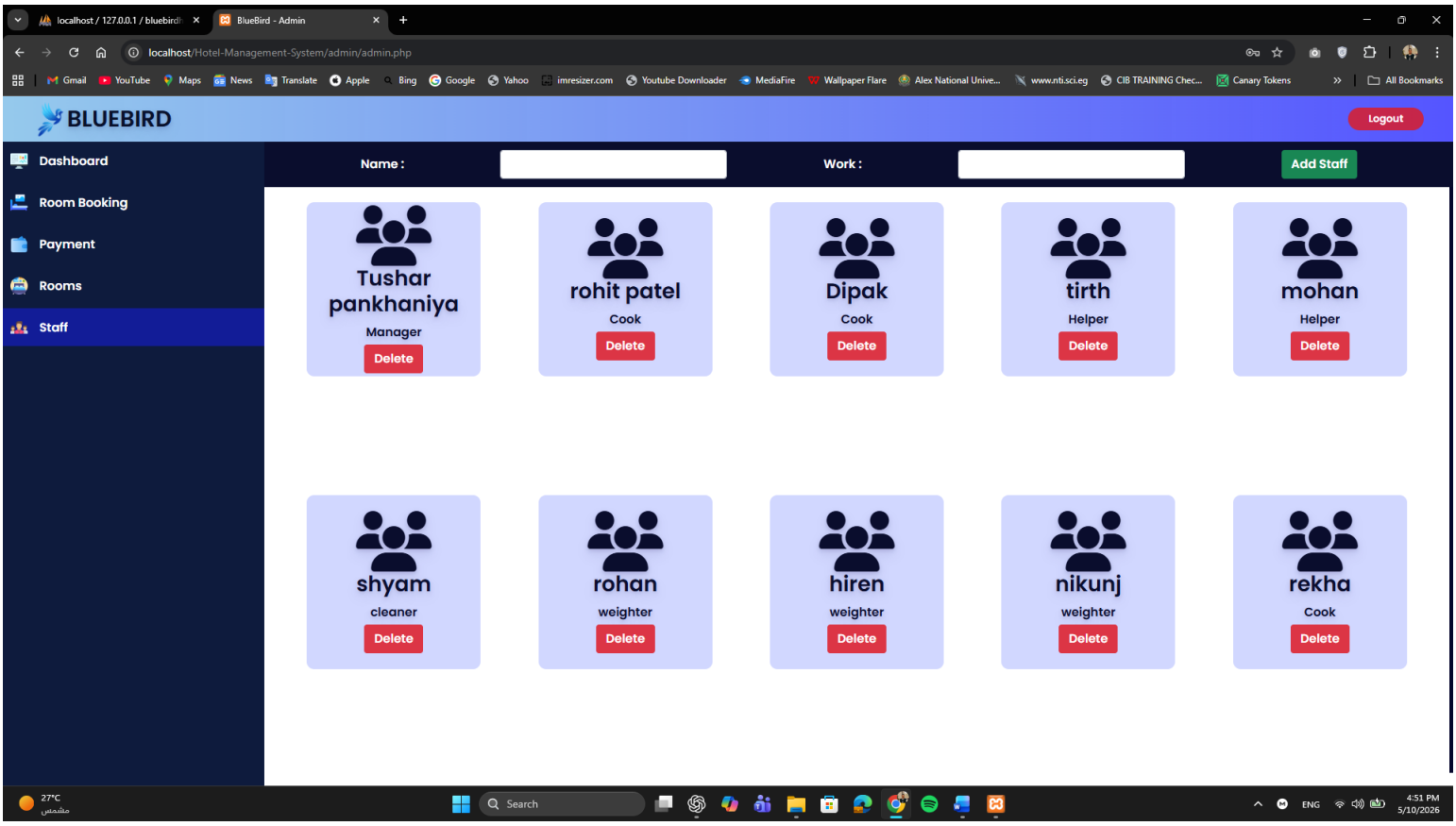
Id	Name	Email	Country	Phone	Type of Room	Type of Bed	No of Room	Meal	Check-In	Check-Out	No of Day	Status	Action
1	Mohamed	Mohamed@Gmail.com	Albania	01208852219	Superior Room	Single	1	Full Board	2026-04-29	2026-05-02	3	Confirm	Edit Delete
2	hossam117	hossam@gmail.com	American Samoa	01011899034	Superior Room	Double	1	Breakfast	2026-05-28	2026-04-28	-30	NotConfirm	Confirm Edit Delete
3	hossam117	hossam@gmail.com	Egypt	01011899034	Superior Room	Double	1		2026-05-12	2026-05-14	2	Confirm	Edit Delete



Id	Name	Room Type	Bed Type	Check In	Check In	No of Day	No of Room	Meal Type	Room Rent	Bed Rent	Meals	Total Bill	Action
1	Mohamed	Superior Room	Single	2026-04-29	2026-05-02	3	1	Full Board	9000.00	90.00	360.00	9450.00	Print Delete
3	hossam117	Superior Room	Double	2026-05-12	2026-05-14	2	1		6000.00	120.00	0.00	6120.00	Print Delete


BLUEBIRD
Logout
Dashboard
Room Booking
Payment
Rooms
Staff
INVOICE
HOTEL BLUE BIRD,
(+91) 9313346569
BLUEBIRD
Mohamed
Invoice # 1
Date 2026-05-02
Item | No of Days | Rate | Quantity | Price
Superior Room | 3 | EGY320 | 1 | EGY9000.00
Single Bed | 3 | EGY3.2 | 1 | EGY90.00
Full Board | 3 | EGY12.8 | 1 | EGY360.00
Total EGY9450.00
Amount Paid EGY0.00
Balance Due EGY9450.00
CONTACT US
Email :- pankhaniyatushar9@gmail.com || Web :- www.bluebird.com || Phone :- +91 9313346569


BLUEBIRD
Logout
Dashboard
Room Booking
Payment
Rooms
Staff
Type of Room :
Type of Bed :
Add Room
Superior Room Single Delete
Superior Room Triple Delete
Superior Room Quad Delete
Deluxe Room Single Delete
Deluxe Room Double Delete
Deluxe Room Triple Delete
Guest House Single Delete
Guest House Double Delete
Guest House Triple Delete
Guest House Quad Delete
Superior Room Quad Delete


localhost / 127.0.0.1 / bluebird
BlueBird - Admin
localhost/Hotel-Management-System/admin/admin.php
BLUEBIRD
Logout
Dashboard
Room Booking
Payment
Rooms
Staff
Name :
Work :
Add Staff
Tushar pankhaniya
Manager
Delete
rohit patel
Cook
Delete
Dipak
Cook
Delete
tirth
Helper
Delete
mohan
Helper
Delete
shyam
cleaner
Delete
rohan
weighter
Delete
hiren
weighter
Delete
nikunj
weighter
Delete
rekha
Cook
Delete
Search
ENG
4:51 PM
5/10/2026

Appendix C. Final Deliverables Checklist

Deliverable	**Status**
Working Software Application	✓
Project Report	✓
Use Case Diagram	✓
Sequence Diagram	✓
Activity Diagram	✓
Class Diagram	✓
State Machine Diagram	✓
Test Cases	✓
Screenshots	✓

--------- END OF REPORT --------

GitHub